

YouTube Remixer Setup Guide

Install, authorize, and post your first remix — step by step, on macOS.

WHAT'S INSIDE

1. Install the tool
2. Create the Google Cloud project
3. Turn on the YouTube Data API
4. Set up the OAuth consent screen
5. Create the OAuth client
6. Finish the install
7. How to use it

Every screenshot below is annotated with exactly what to click or type. Steps 1–39 are one-time setup; after that you only ever run `rmx`.

Part 1 Install the tool

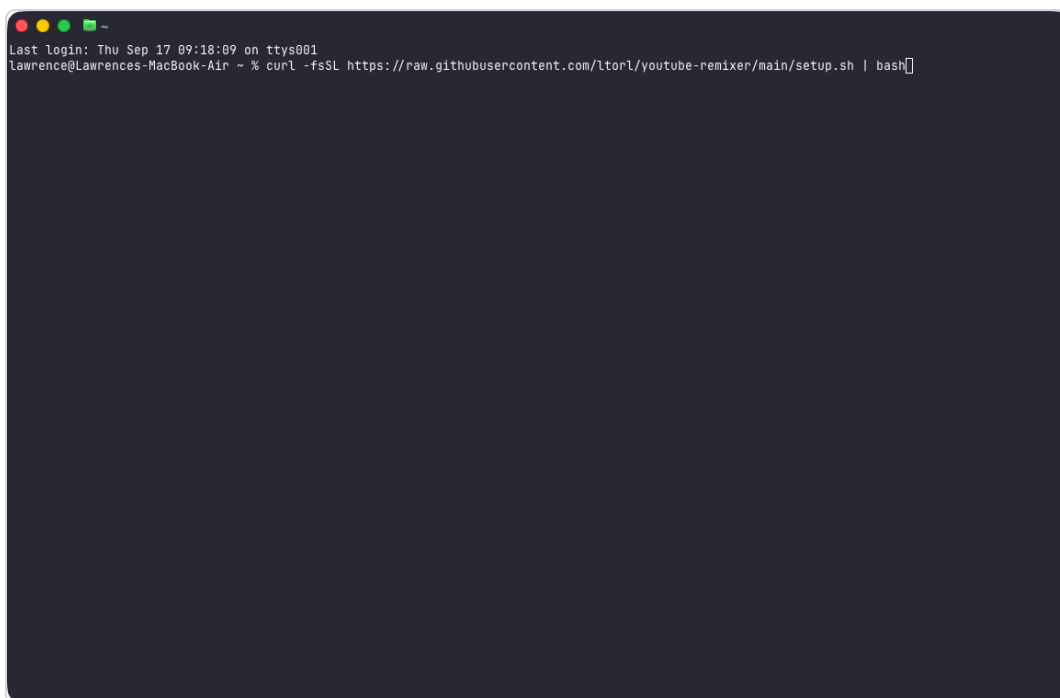
Before you start

You need a Mac, a Google account, and the YouTube channel you want to post to signed in to that same account. Setup takes about 15 minutes and only happens once.

1 Run the one-line installer

Open **Terminal** (Spotlight → type *Terminal* → Enter). Click once in the black window, paste the command below, then press **Return**.

- Command to paste: `curl -fsSL https://raw.githubusercontent.com/ltorl/youtube-remixer/main/setup.sh | bash`
- Leave this Terminal window open for the rest of the guide — you come back to it at Step 25.



2 Wait about 30 seconds

Don't type anything. The installer downloads and builds the tools, then opens the Google Cloud Console in your browser by itself.

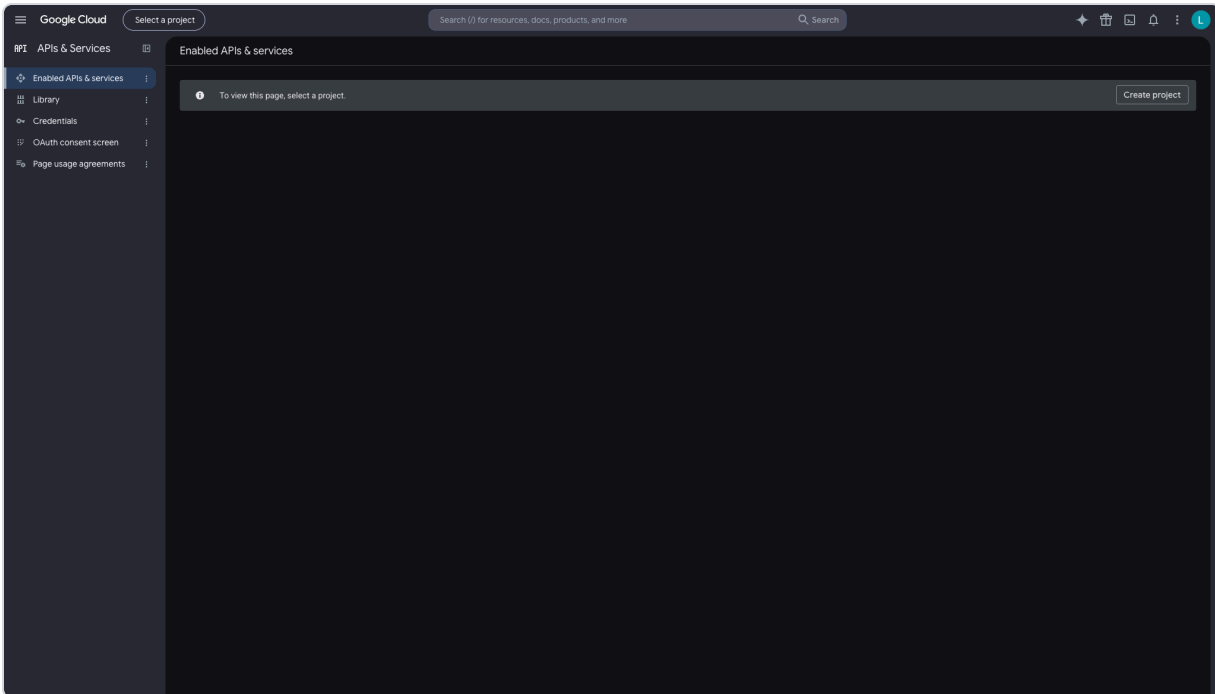
- If the browser doesn't open after a minute, go to `console.cloud.google.com/apis/dashboard` manually.

Part 2 Create the Google Cloud project

3 Create a Google Cloud project

In the browser, click **Select a project** in the blue top bar (or the **Create project** button on the right of the info banner).

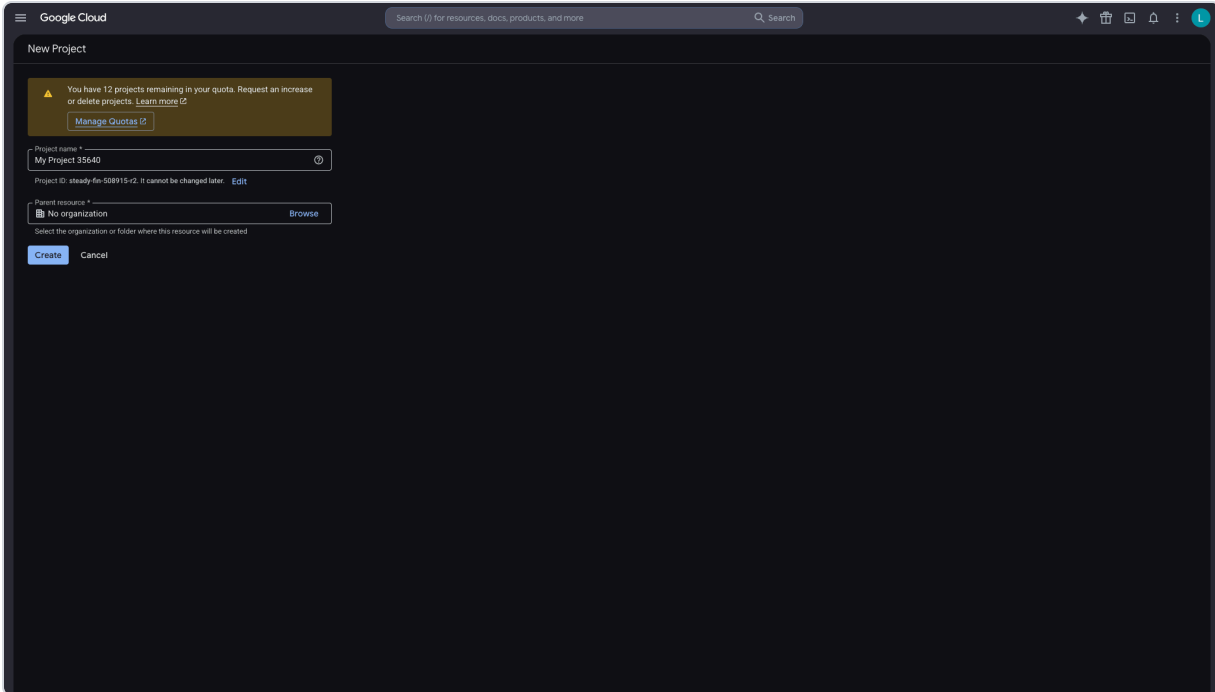
- Sign in first with the Google account that owns the YouTube channel you'll post to.



4 Name the project

Click the **Project name** box and type a name — e.g. `youtube-remixer`. Leave **Location / Parent resource** as *No organization*. Click the blue **Create** button.

- The Project ID underneath is generated automatically — you don't need it.



The screenshot shows the 'New Project' form in the Google Cloud console. At the top, there's a header with the Google Cloud logo and a search bar. Below the header, a yellow warning box states: 'You have 12 projects remaining in your quota. Request an increase or delete projects. Learn more (2)'. Below this, the 'Project name' field is set to 'My Project 35640'. The 'Project ID' is 'steady-fm-508915-r2', with a note that it cannot be changed later. The 'Parent resource' is set to 'No organization'. At the bottom, there are 'Create' and 'Cancel' buttons.

Google Cloud

Search (0) for resources, docs, products, and more

Search

New Project

You have 12 projects remaining in your quota. Request an increase or delete projects. Learn more (2)

Manage Quotas (2)

Project name *

My Project 35640

Project ID: steady-fm-508915-r2. It cannot be changed later. Edit

Parent resource *

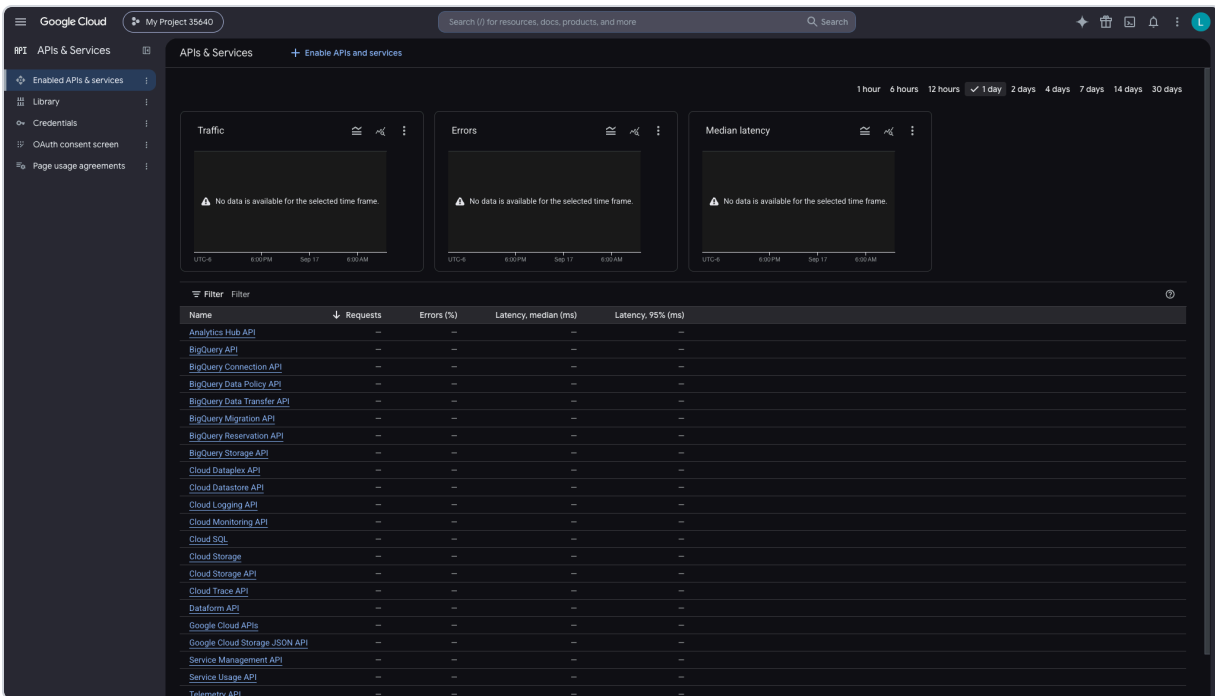
No organization

Select the organization or folder where this resource will be created

Create Cancel

5 Project is ready

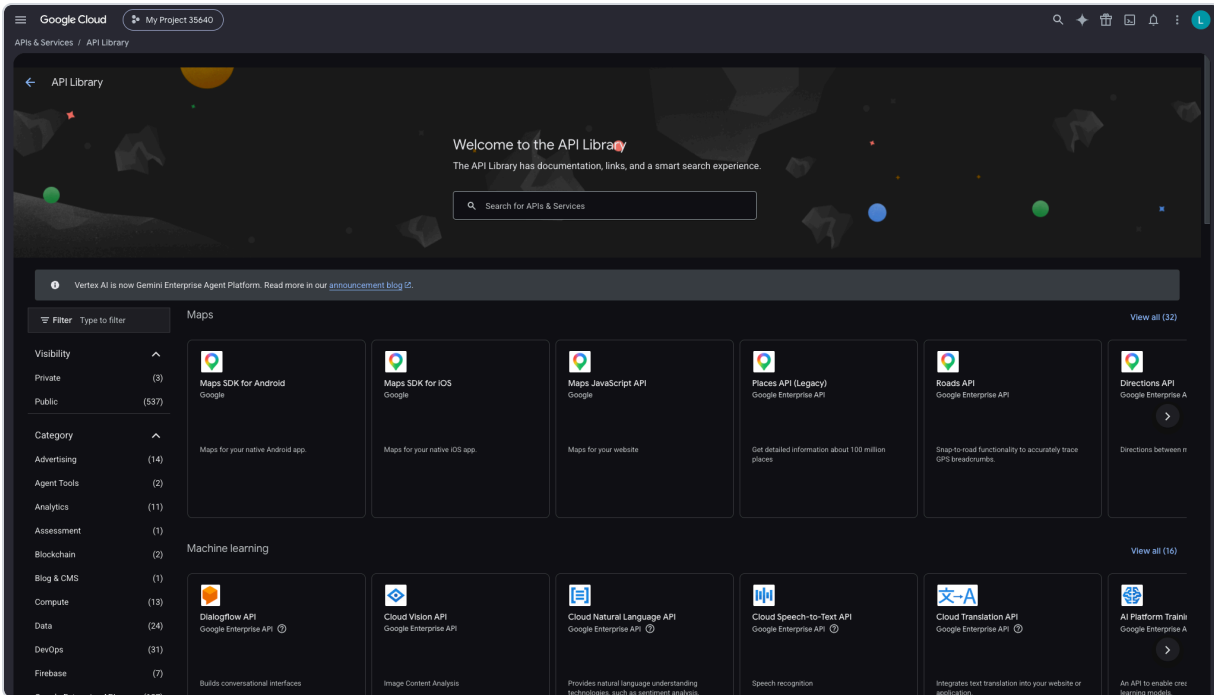
Nothing to click yet. Confirm the project name now shows in the top bar next to the Google Cloud logo. If it shows a different project, click that chip and pick the one you just made.



Part 3 Turn on the YouTube Data API

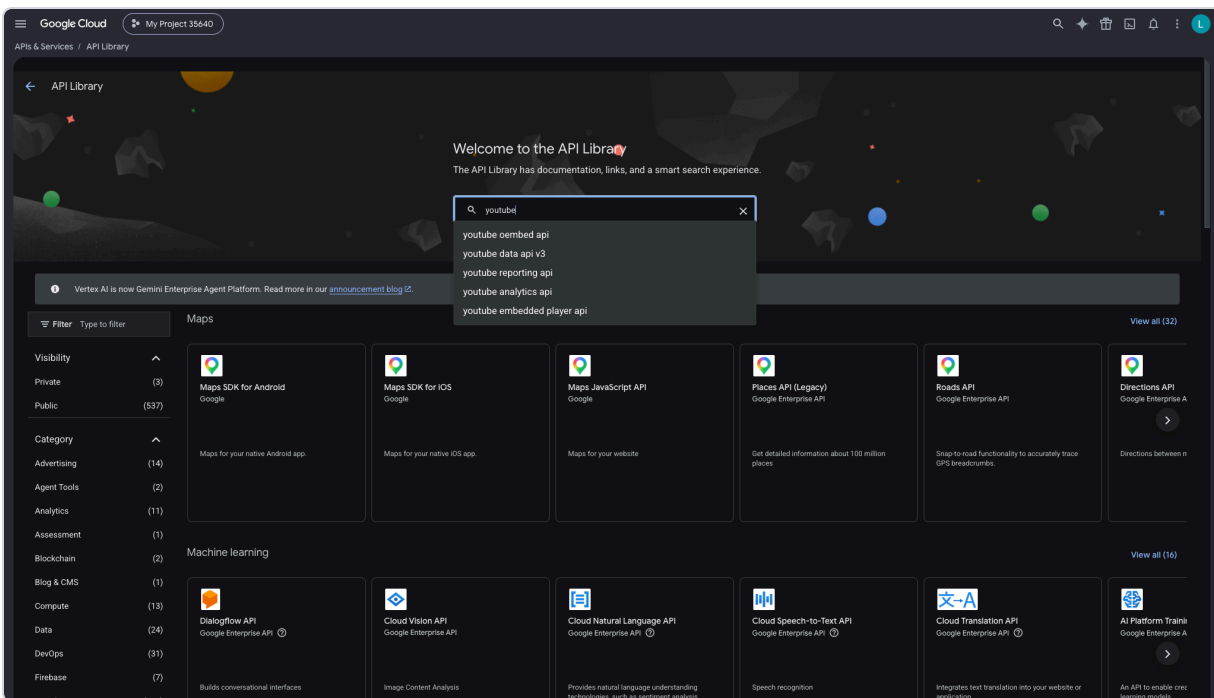
6 Open the API Library

In the left sidebar, click **Library**.



7 Search for the YouTube API

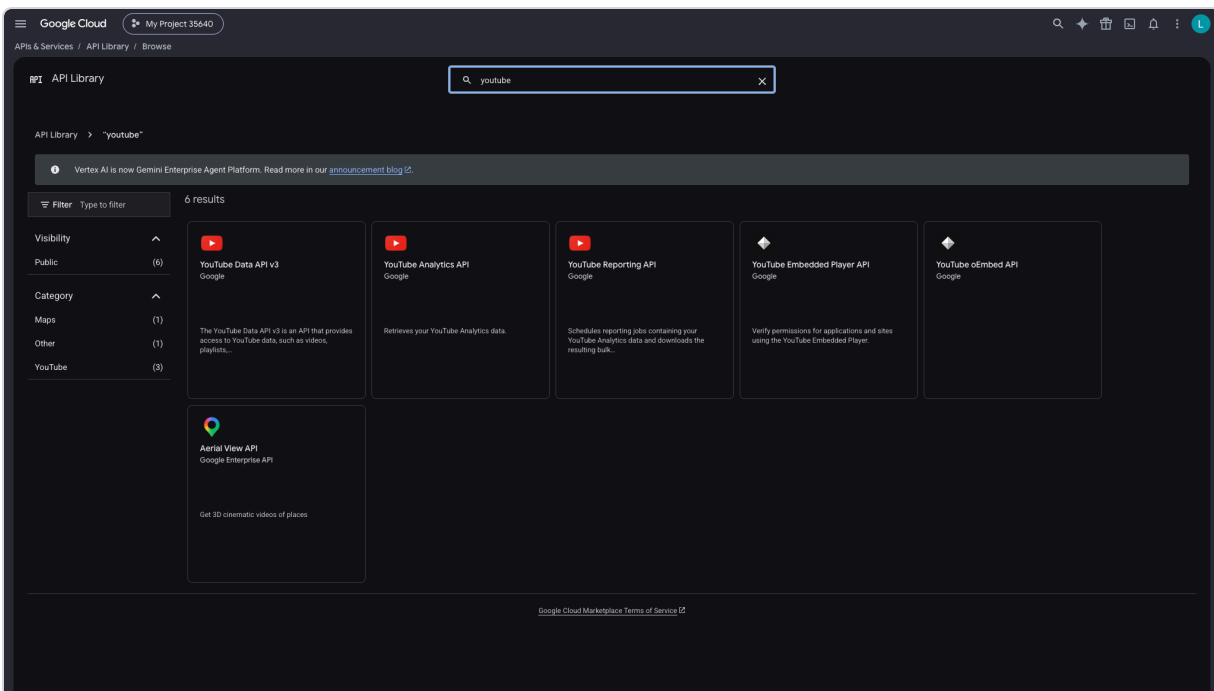
Click the **Search for APIs & Services** box in the middle of the page and type `youtube`. Press **Return** (don't pick a suggestion).



8 Pick YouTube Data API v3

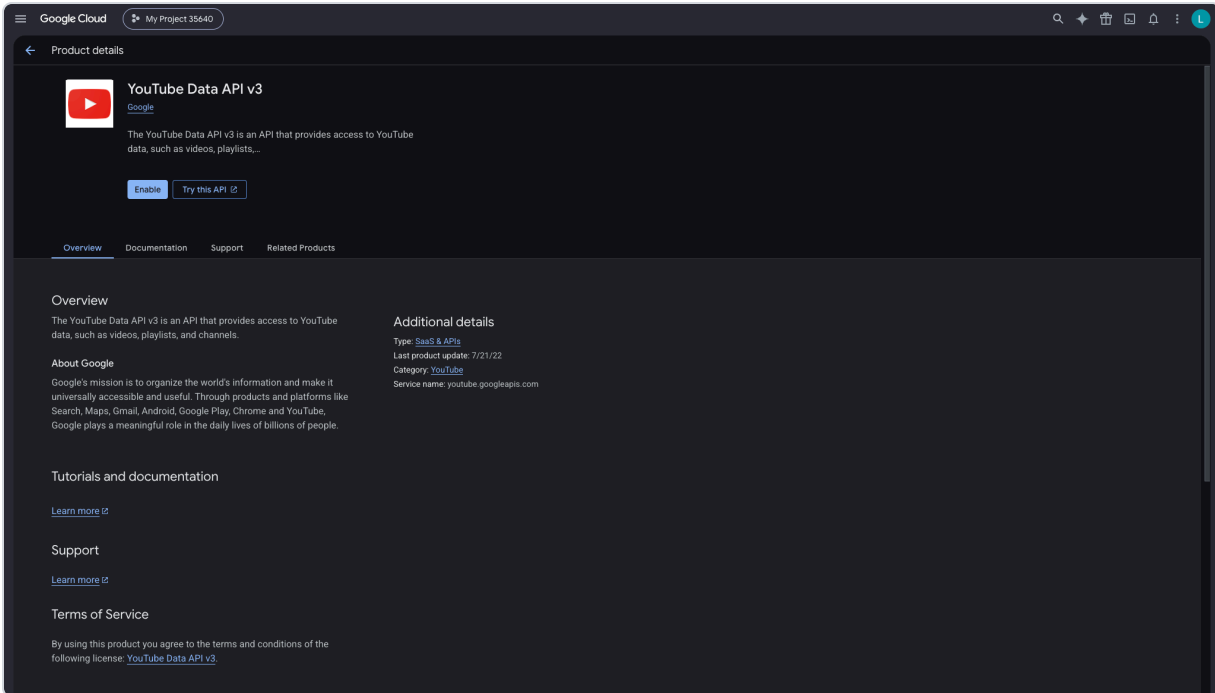
In the results, click the first card: **YouTube Data API v3**.

- Make sure it's *Data API v3* — not Analytics, Reporting, or Embedded Player.



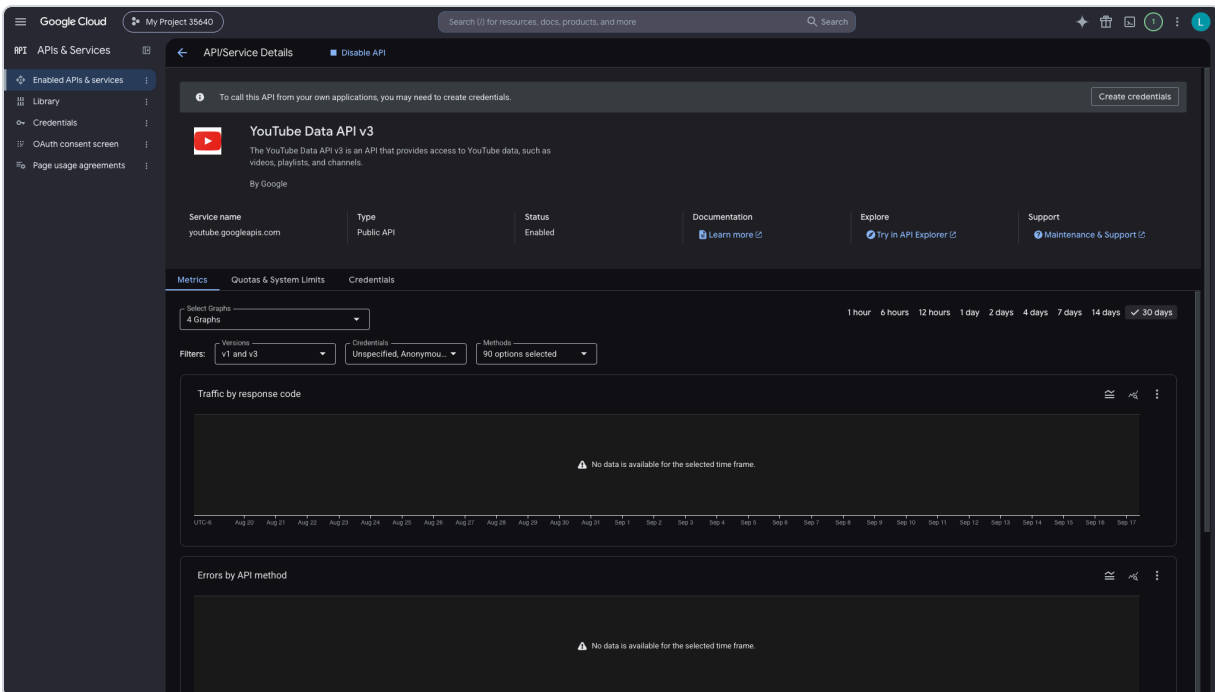
9 Enable it

Click the blue **Enable** button under the API title.



10 Enabled — confirm

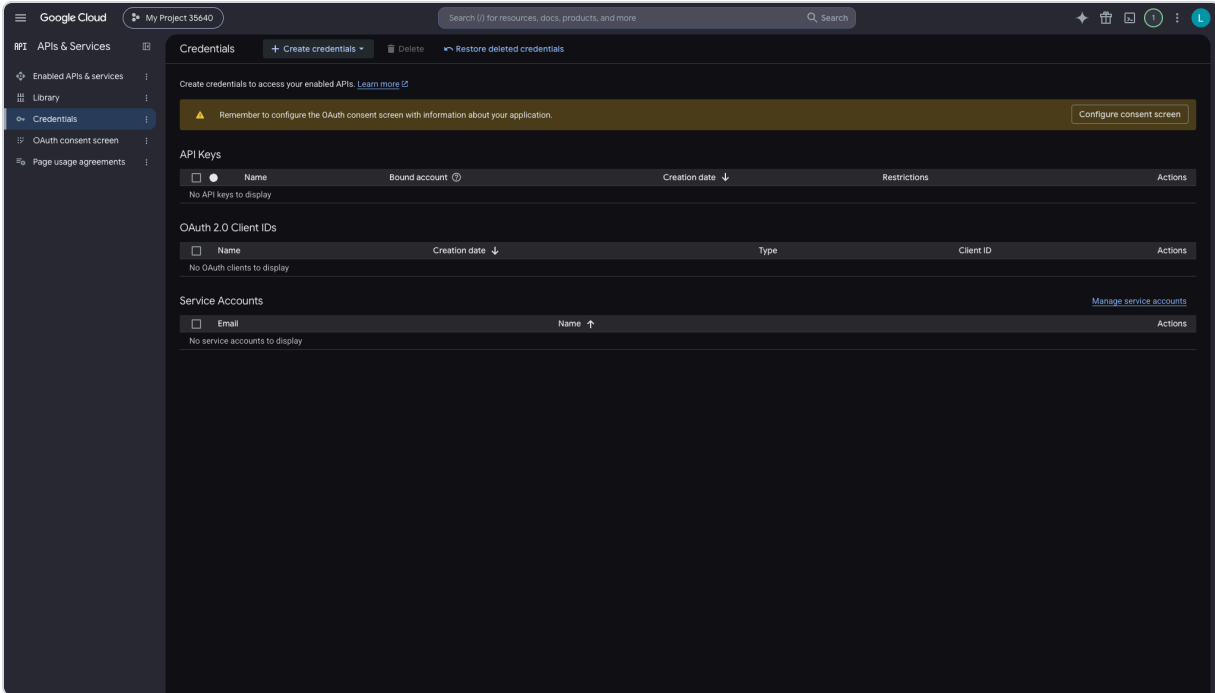
Check that **Status** reads *Enabled*. Nothing else to do on this screen.



Part 4 Set up the OAuth consent screen

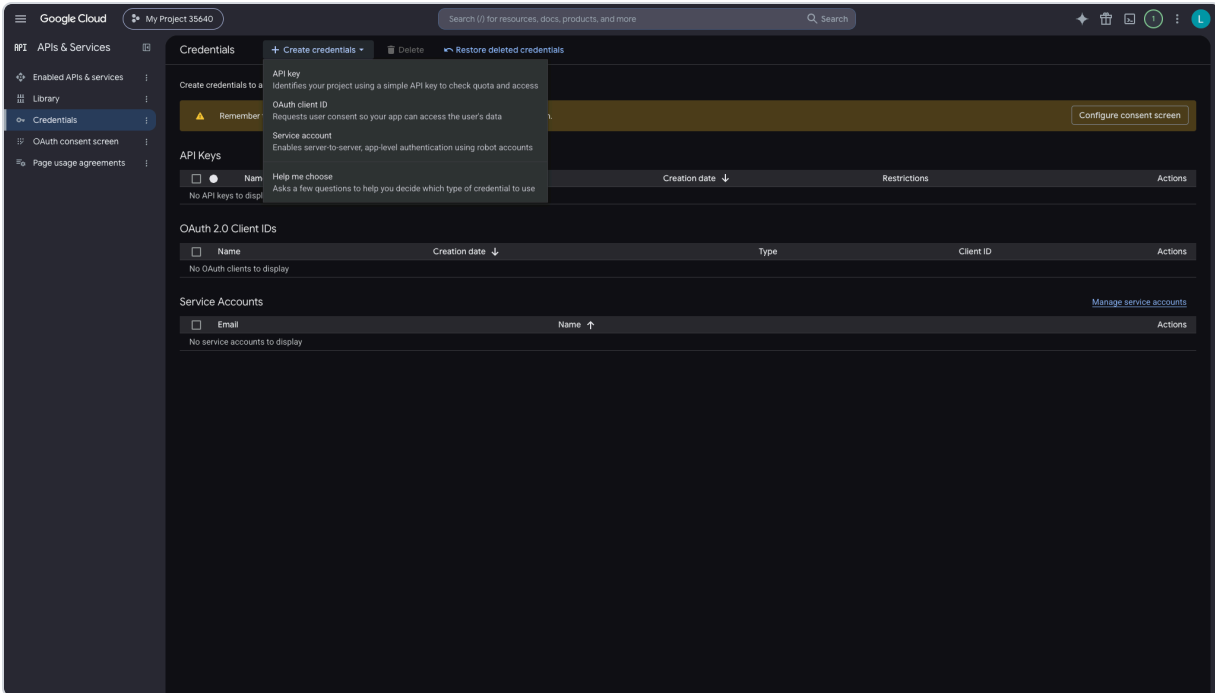
11 Go to Credentials

In the left sidebar, click **Credentials**.



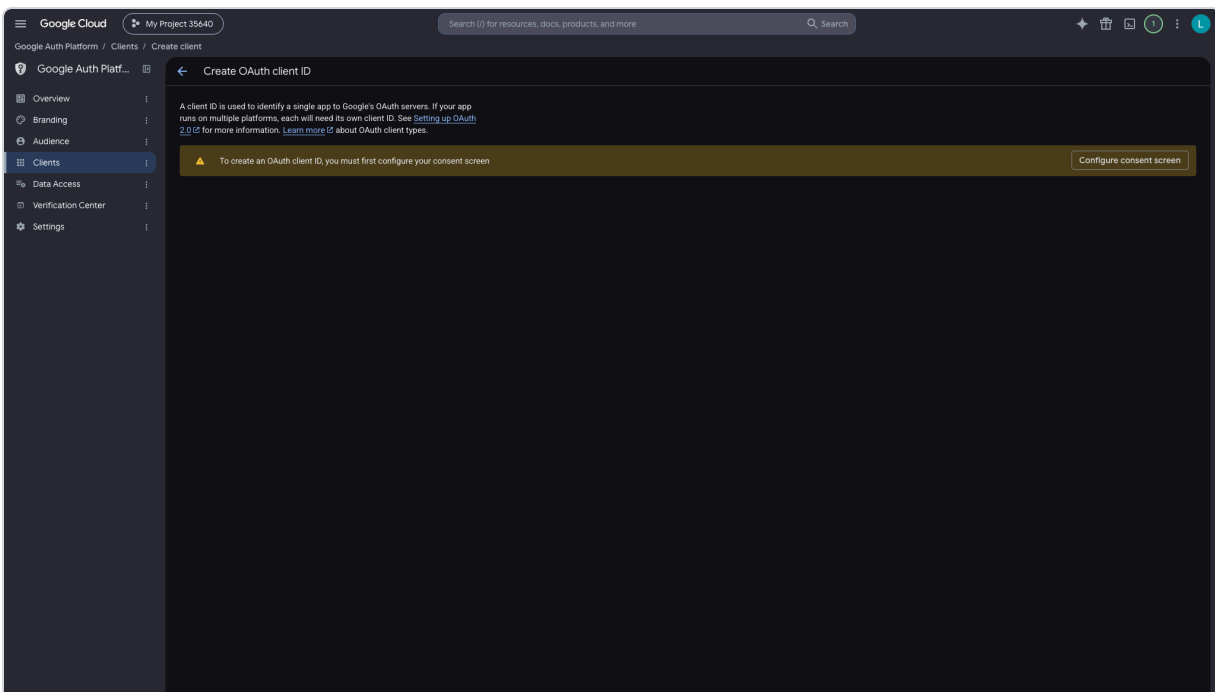
12 Start a credential

Click **+ Create credentials** at the top, then choose **OAuth client ID** from the dropdown.



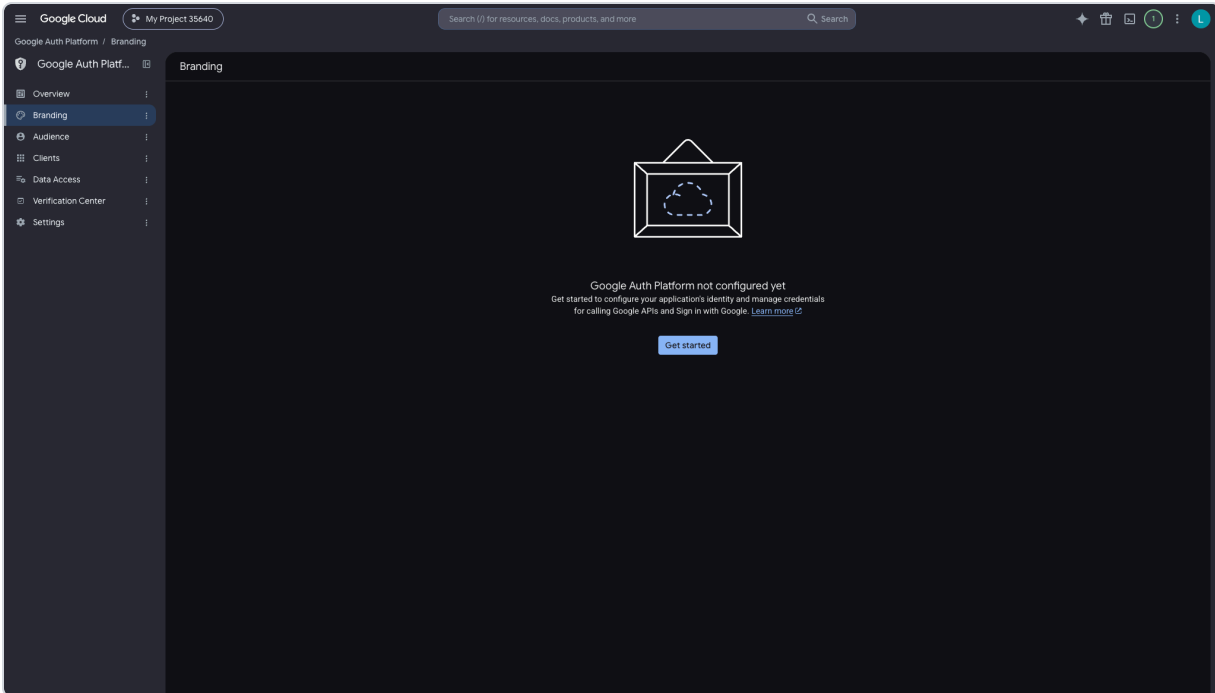
13 Consent screen required first

Google blocks you until the consent screen exists. Click the white **Configure consent screen** button on the right of the yellow banner.



14 Get started with the Auth Platform

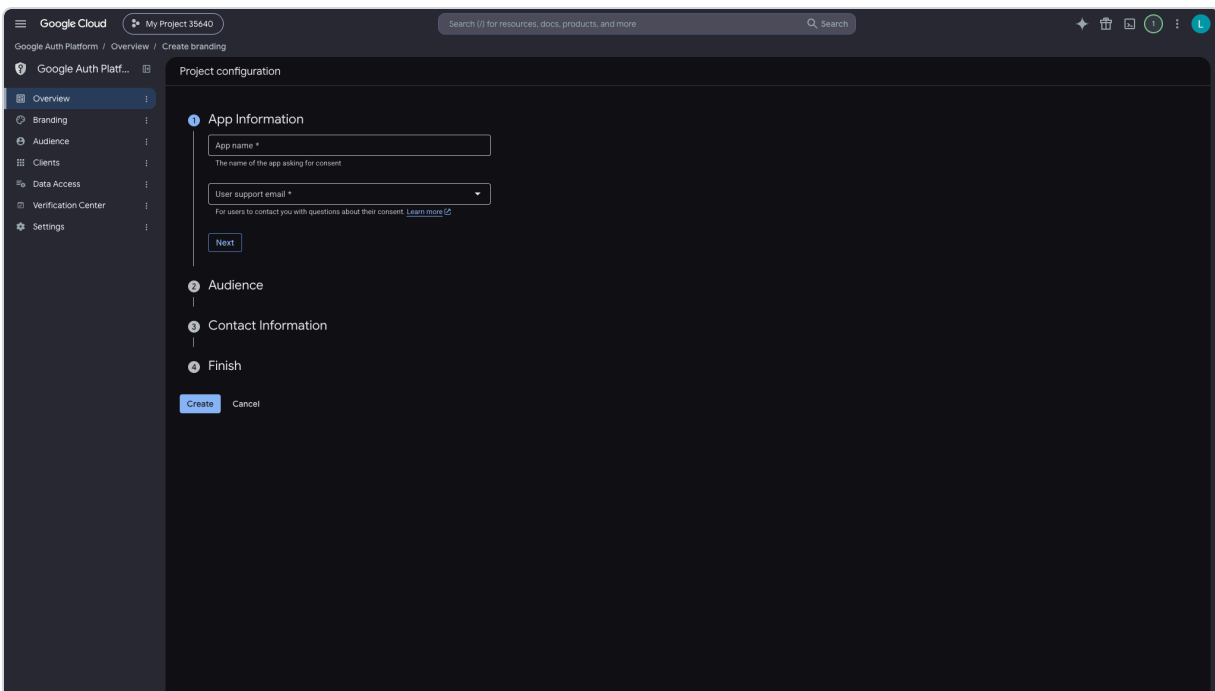
Click the blue **Get started** button in the middle of the page.



15 Step 1 — App Information

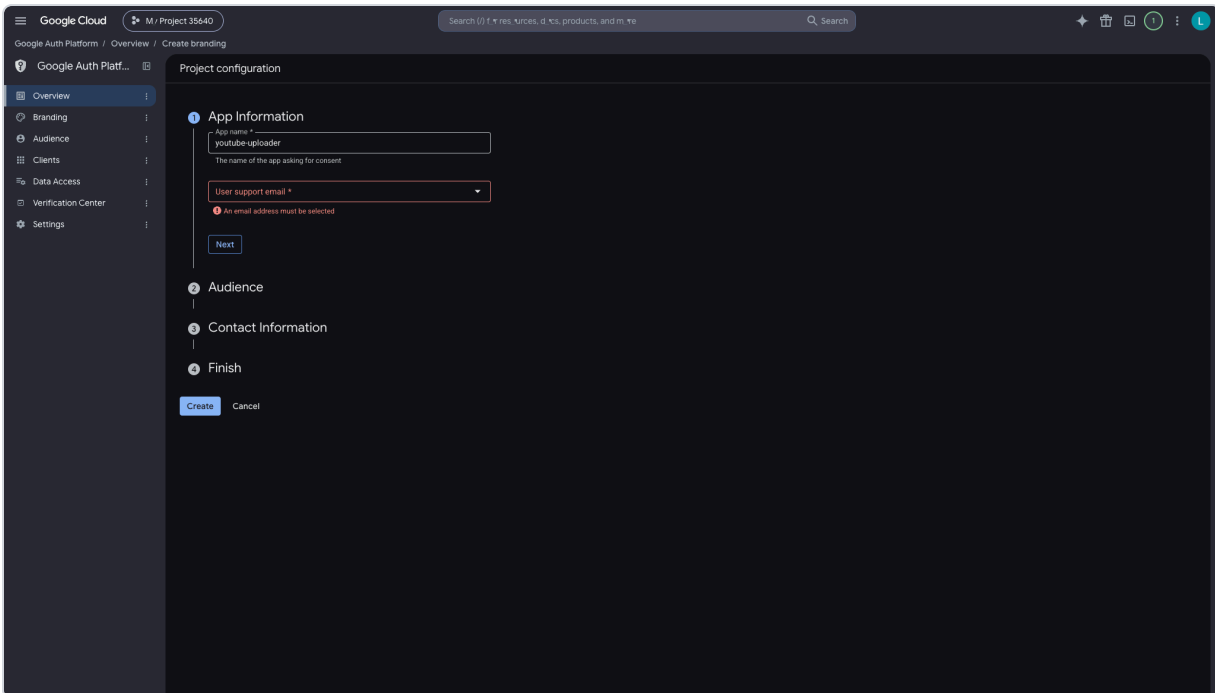
Click the **App name** field and type a name, e.g. `youtube-uploader`. Then click the **User support email** dropdown and pick your own email address.

- Both fields are required — the Next button won't advance until each is filled.



16 Choose the support email

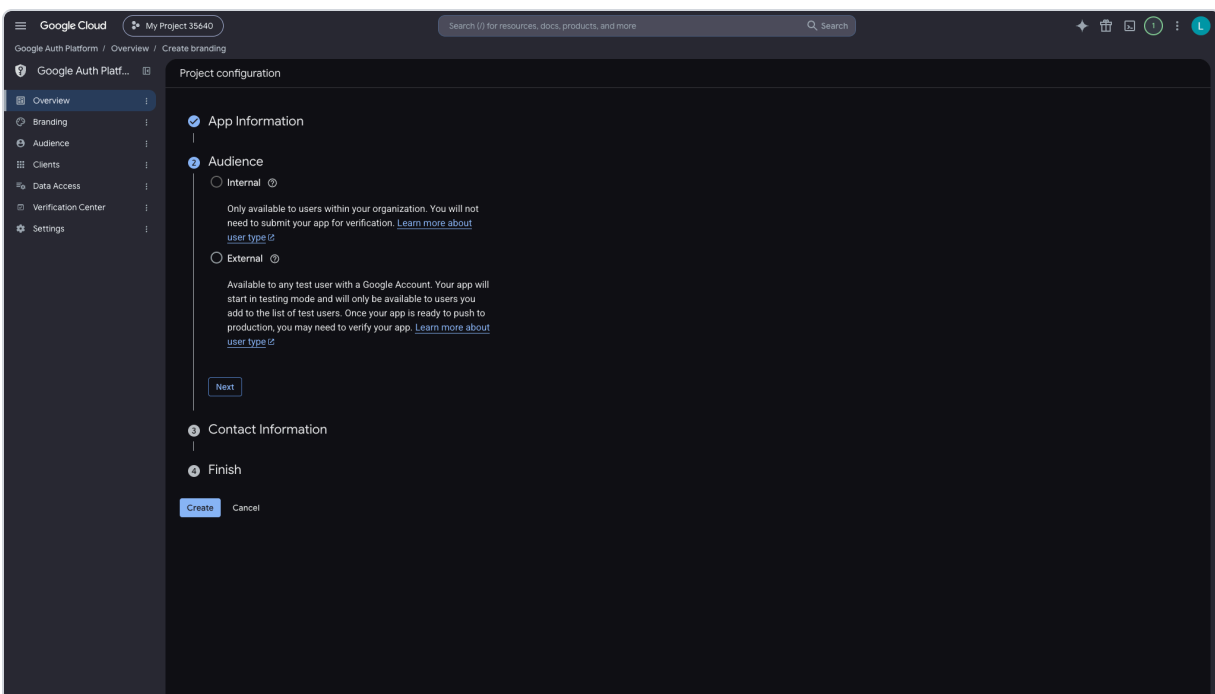
If you see the red “An email address must be selected” warning, click the **User support email** dropdown and select your address, then click **Next**.



17 Step 2 — Audience

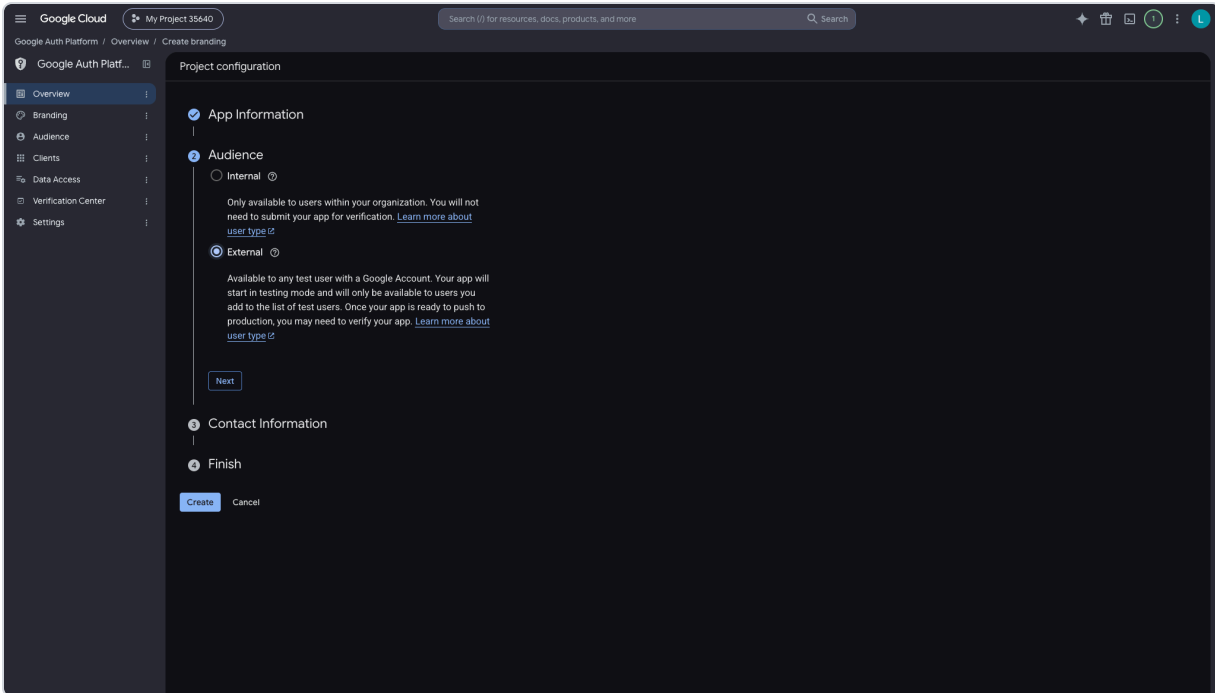
Click the **External** radio button (the lower of the two).

- **Internal** only works on Google Workspace org accounts — for a personal Gmail account it will be greyed out or will fail.



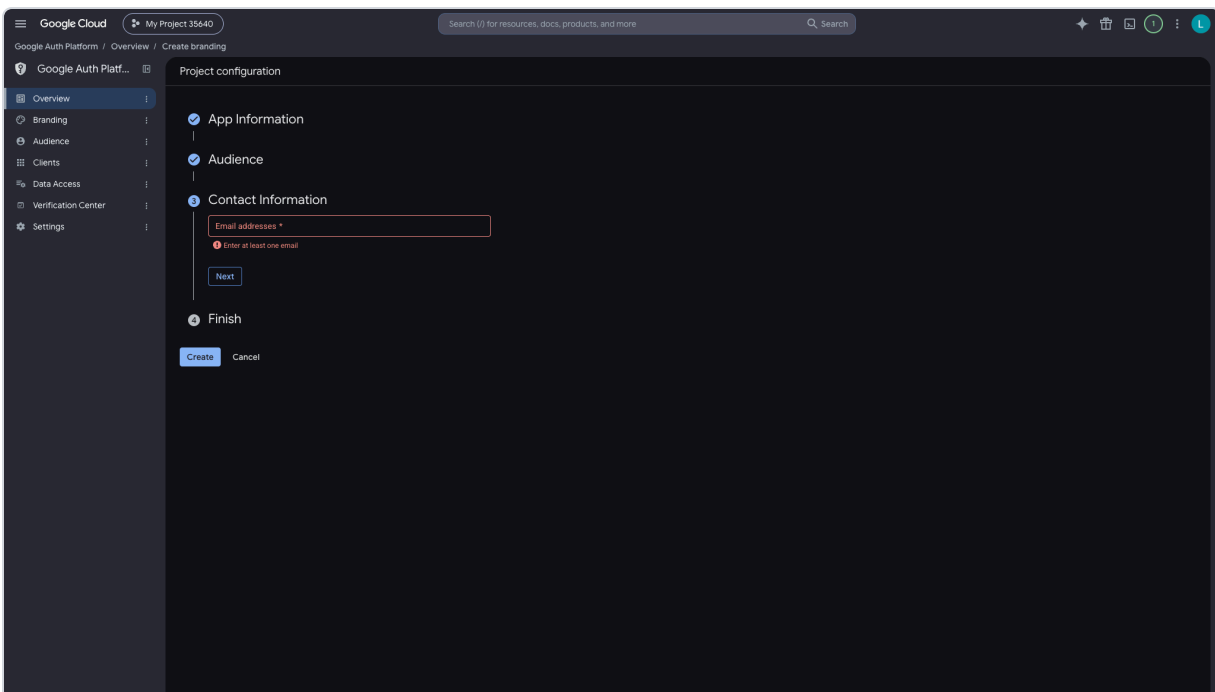
18 External selected

With **External** filled in, click **Next**.



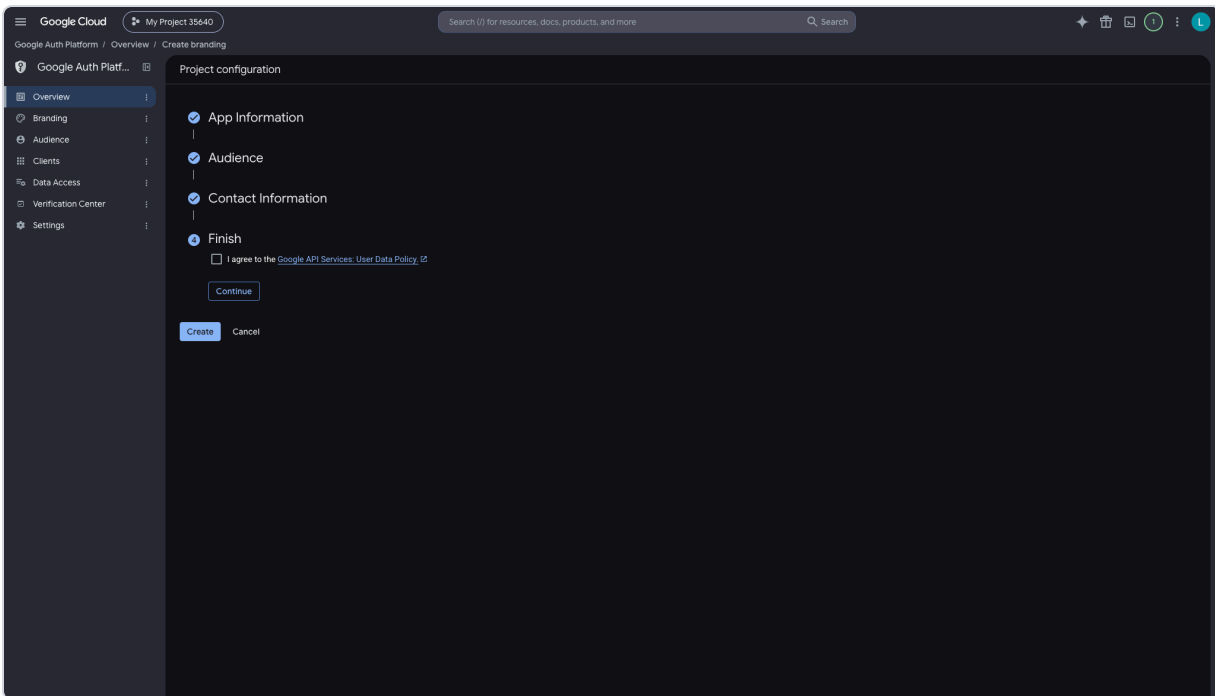
19 Step 3 — Contact Information

Click the **Email addresses** field, type your own email address, press **Return** so it becomes a chip, then click **Next**.



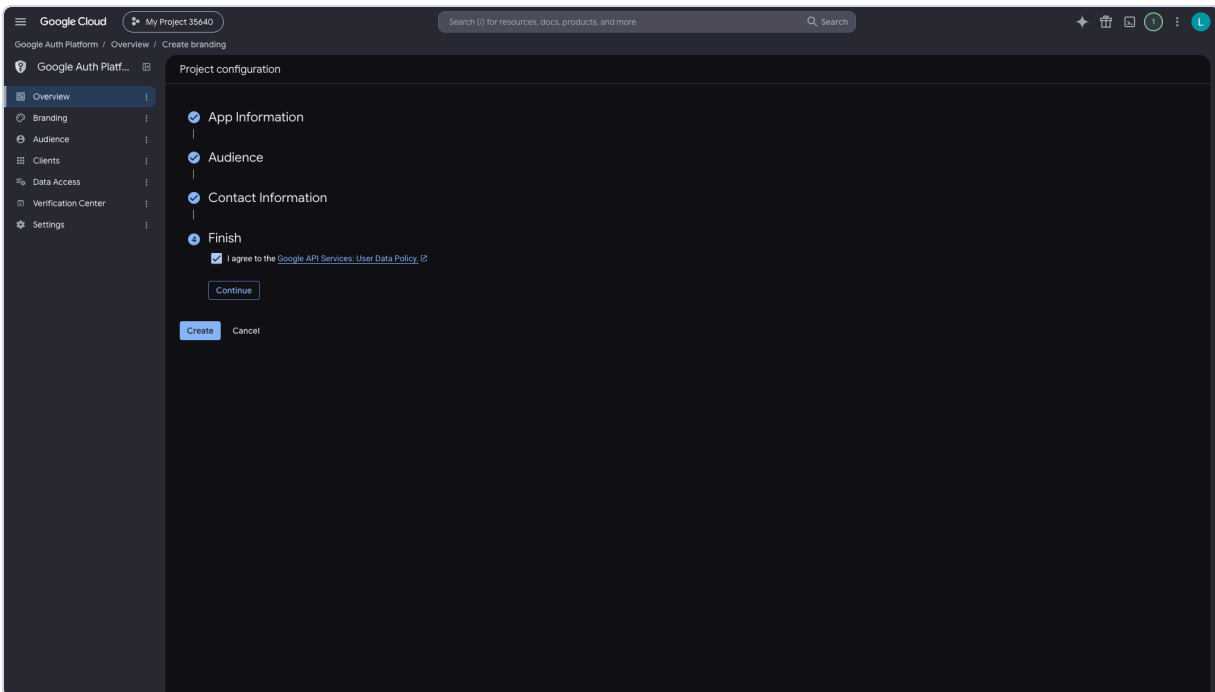
20 Step 4 — Finish

Click the checkbox next to **I agree to the Google API Services: User Data Policy**.



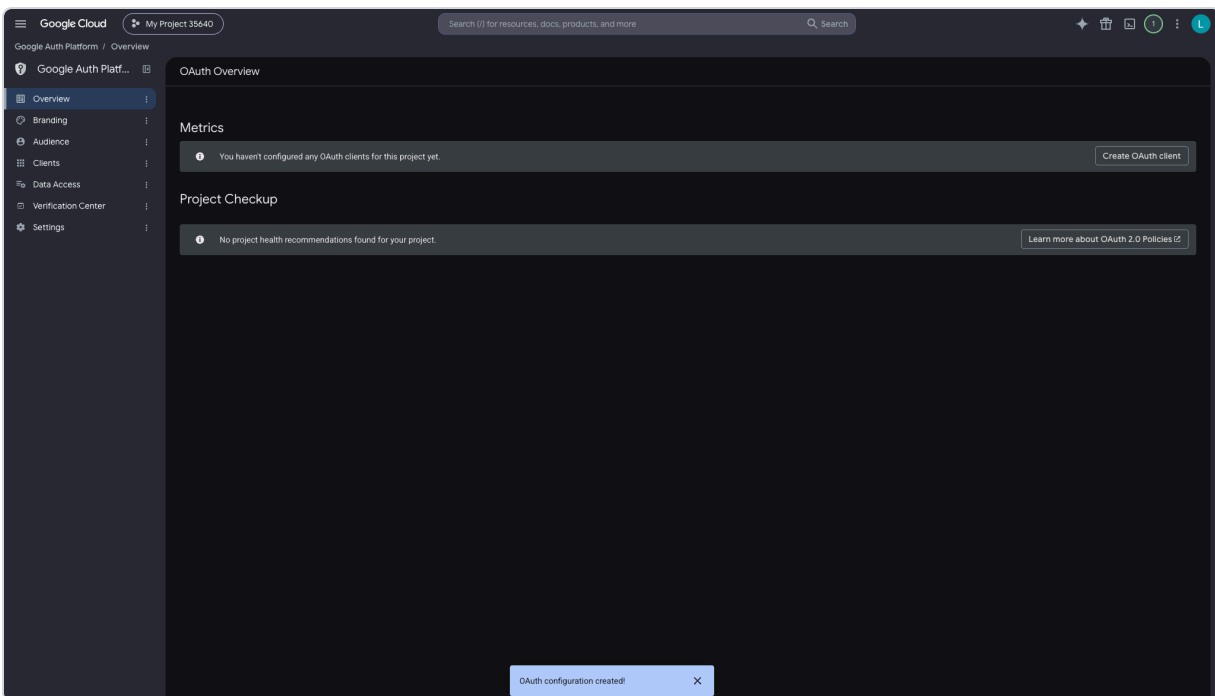
21 Agree and continue

With the box ticked, click **Continue**, then click the blue **Create** button at the bottom.



22 Consent screen created

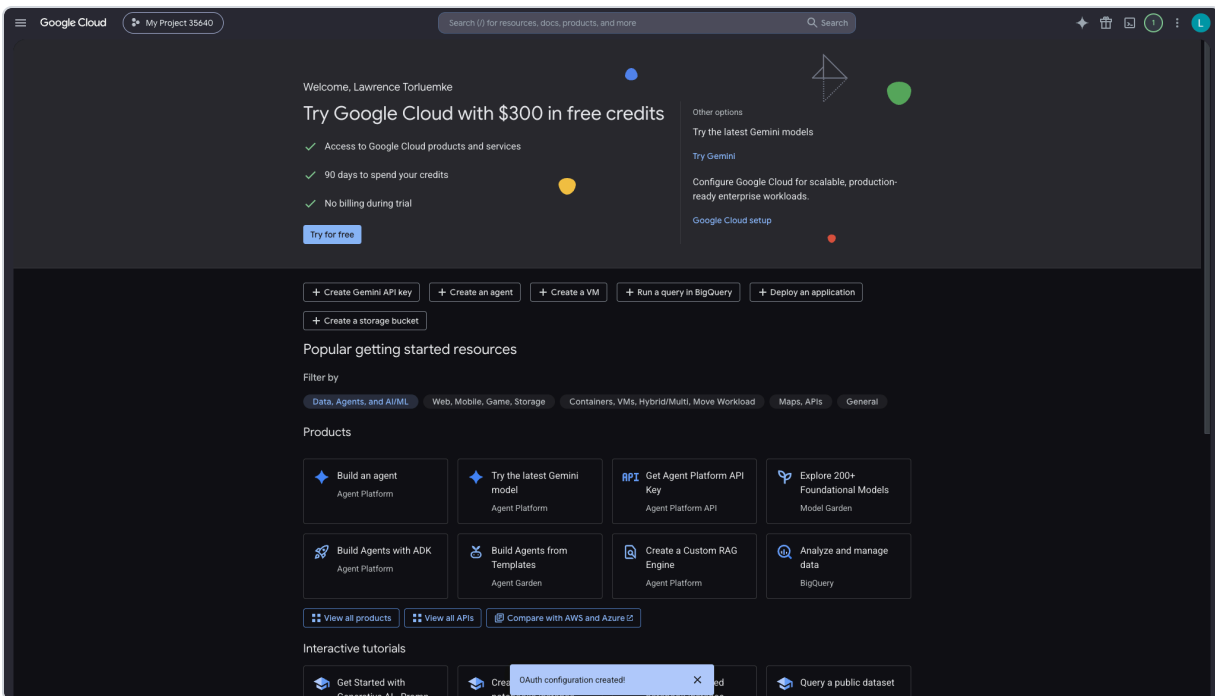
You'll land on **OAuth Overview** with a blue *“OAuth configuration created!”* toast at the bottom. Nothing to click here — head back to Credentials next.



Part 5 Create the OAuth client

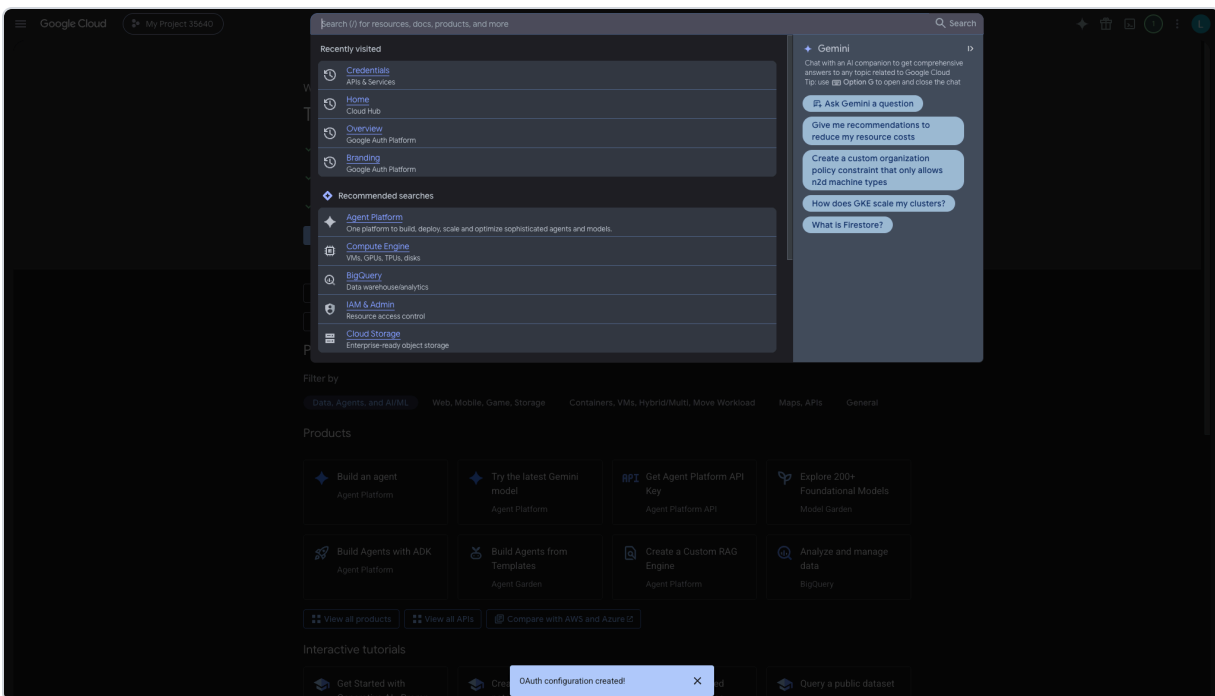
23 Back to the console home

If you get bounced to the Cloud home page, use the search bar at the top to get back — click it once.



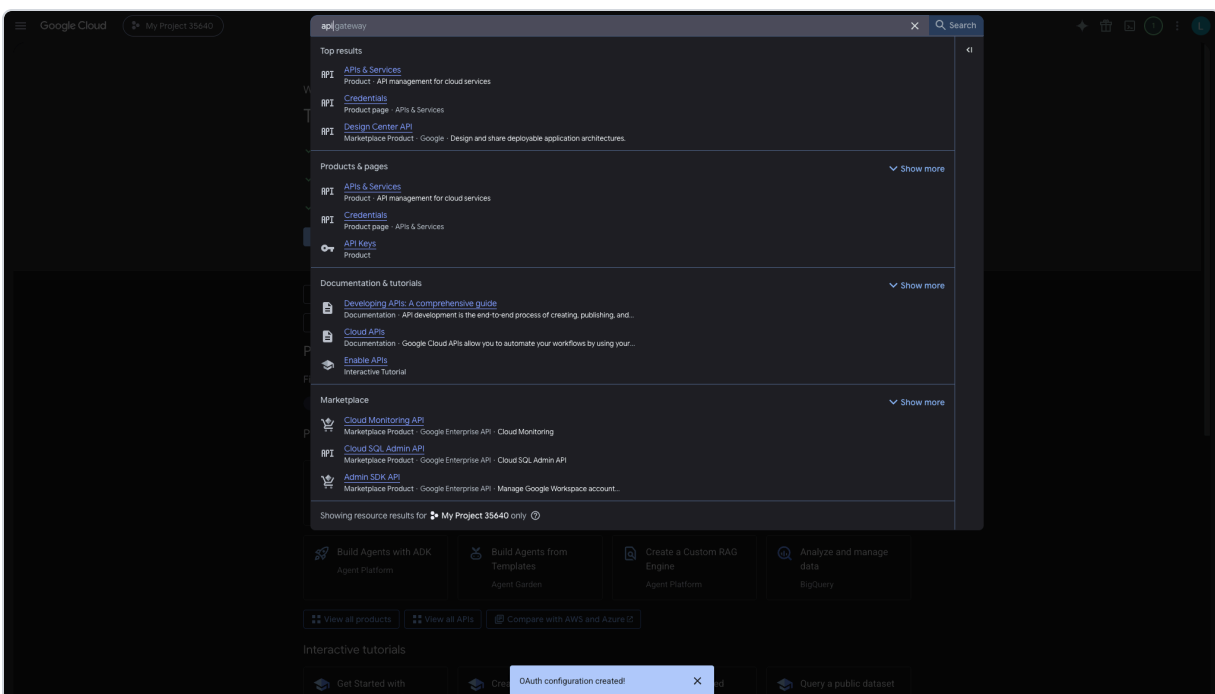
24 Search bar opens

The dropdown shows recently visited pages. **Credentials — APIs & Services** is usually the first entry — you can click it directly and skip to Step 26.



25 Or type to find it

Type `api` in the search bar and click **APIs & Services** under *Top results*.



26 APIs & Services dashboard

In the left sidebar, click **Credentials**.

The screenshot shows the Google Cloud APIs & Services dashboard. The left sidebar has 'Credentials' selected. The main area displays three charts (Traffic, Errors, Median latency) with 'No data is available for the selected time frame.' Below the charts is a table of enabled APIs. A notification at the bottom says 'OAuth configuration created!'.

Name	Requests	Errors (%)	Latency, median (ms)	Latency, 95% (ms)
Analytics Hub API	--	--	--	--
BigQuery API	--	--	--	--
BigQuery Connection API	--	--	--	--
BigQuery Data Policy API	--	--	--	--
BigQuery Data Transfer API	--	--	--	--
BigQuery Migration API	--	--	--	--
BigQuery Reservation API	--	--	--	--
BigQuery Storage API	--	--	--	--
Cloud Dataplex API	--	--	--	--
Cloud Datastore API	--	--	--	--
Cloud Logging API	--	--	--	--
Cloud Monitoring API	--	--	--	--
Cloud SQL	--	--	--	--
Cloud Storage	--	--	--	--
Cloud Storage API	--	--	--	--
Cloud Trace API	--	--	--	--
Dataform API	--	--	--	--
Google Cloud APIs	--	--	--	--
Google Cloud Storage JSON API	--	--	--	--
Service Management API	--	--	--	--
Service Usage API	--	--	--	--
Telemetry API	--	--	--	--

27 Credentials page

Confirm **OAuth 2.0 Client IDs** is still empty — you're about to add one.

The screenshot shows the Google Cloud Credentials page. The left sidebar has 'Credentials' selected. The main area displays three sections: API Keys, OAuth 2.0 Client IDs, and Service Accounts. All sections are empty. A notification at the bottom says 'OAuth configuration created!'.

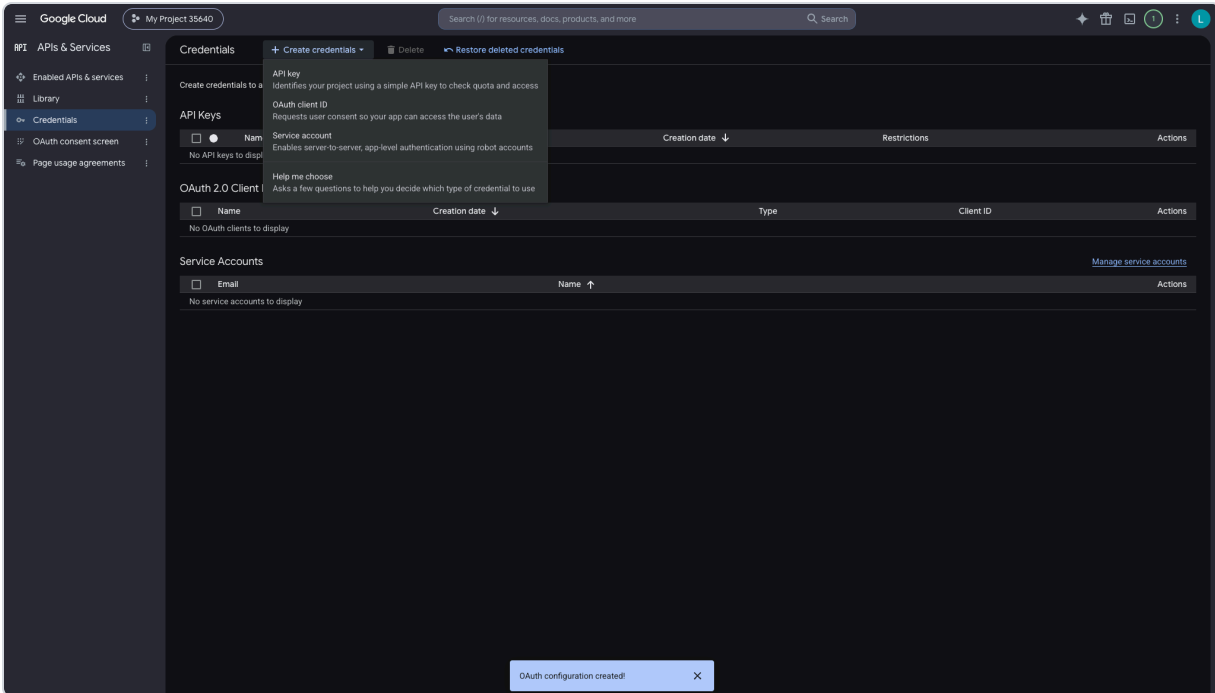
Name	Bound account	Creation date	Restrictions	Actions
No API keys to display				

Name	Creation date	Type	Client ID	Actions
No OAuth clients to display				

Email	Name	Actions
No service accounts to display		

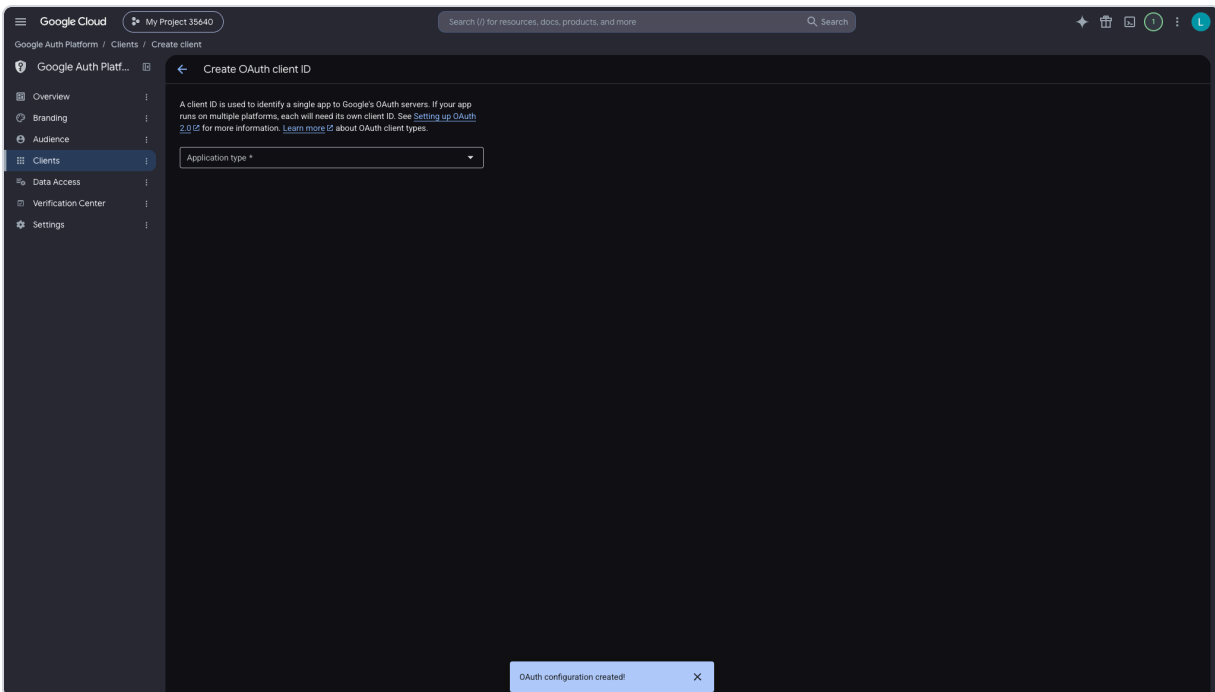
28 Create the OAuth client

Click + **Create credentials** → **OAuth client ID**.



29 Choose the application type

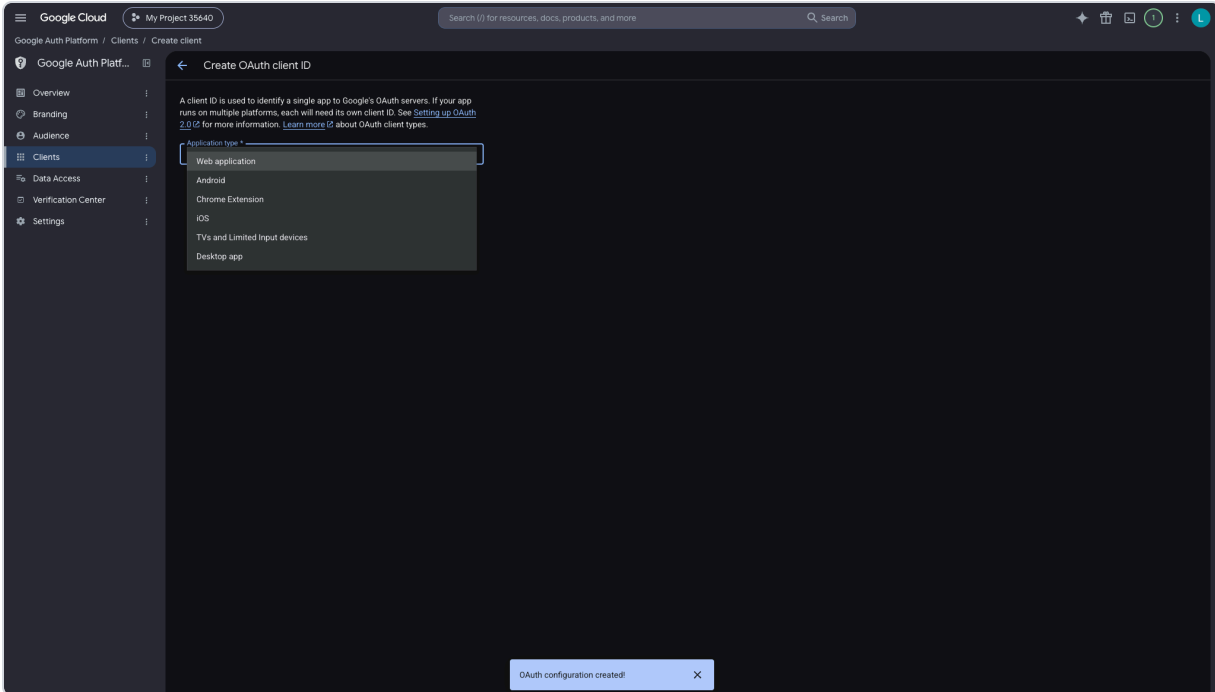
Click the **Application type** dropdown.



30 Select Web application

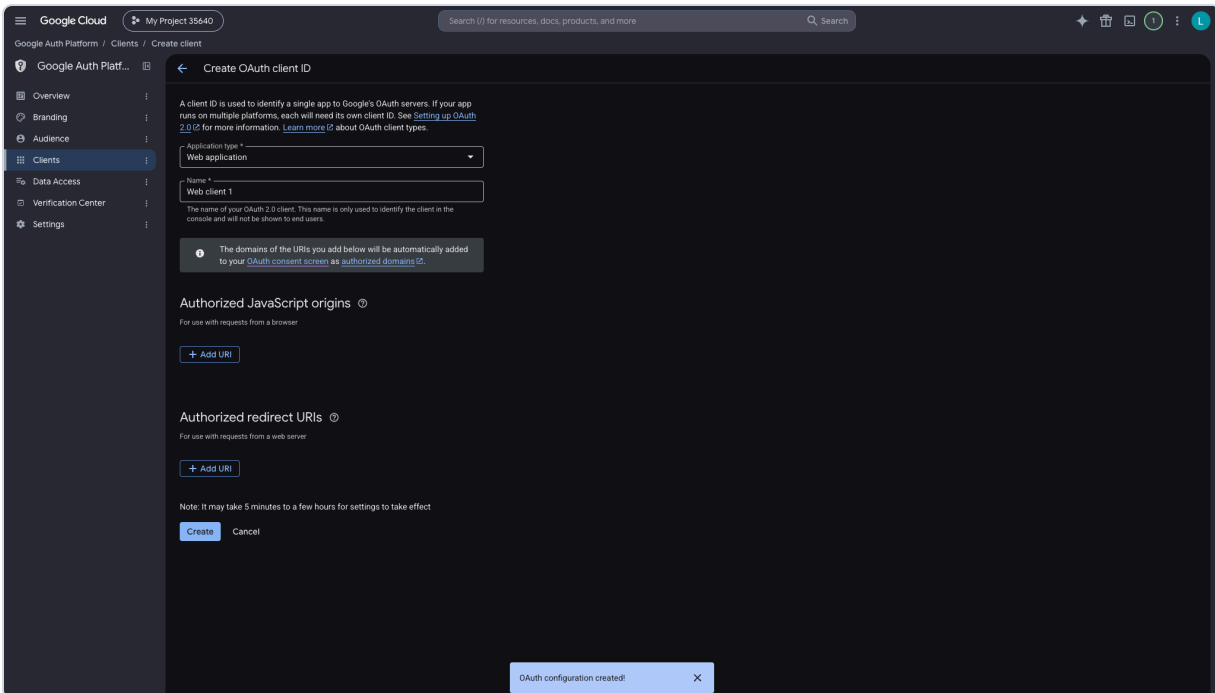
In the open list, click **Web application** (the first option).

- Yes — *Web application*, not *Desktop app*. The installer patches the tool to use a local loopback redirect, which needs the Web type.



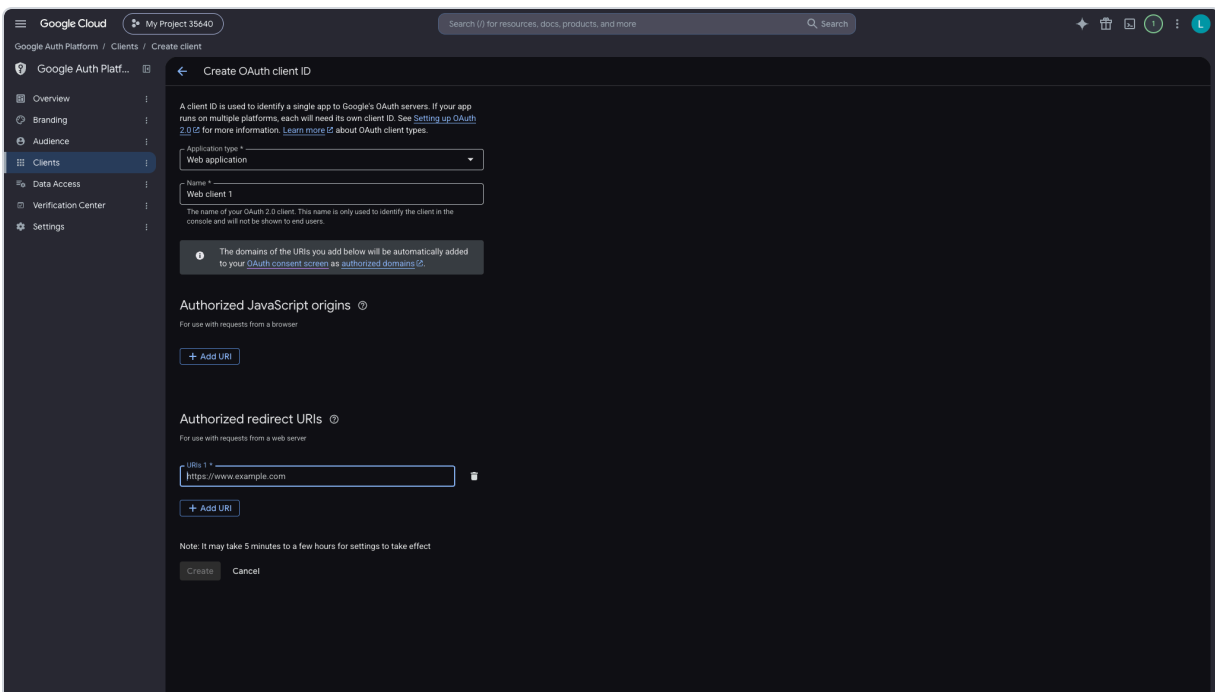
31 Name the client

Leave **Name** as `Web client 1` or type your own. Skip **Authorized JavaScript origins** entirely. Scroll to **Authorized redirect URIs** and click **+ Add URI**.



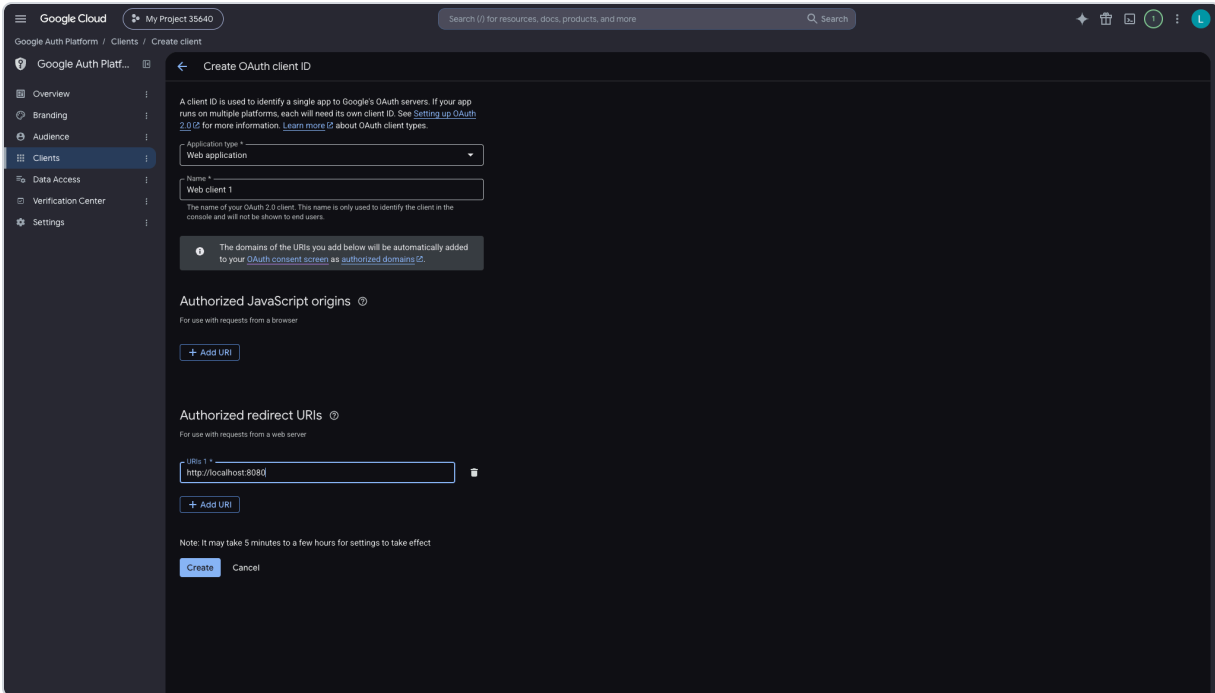
32 Redirect URI 1

Click the **URIs 1** box that appears.



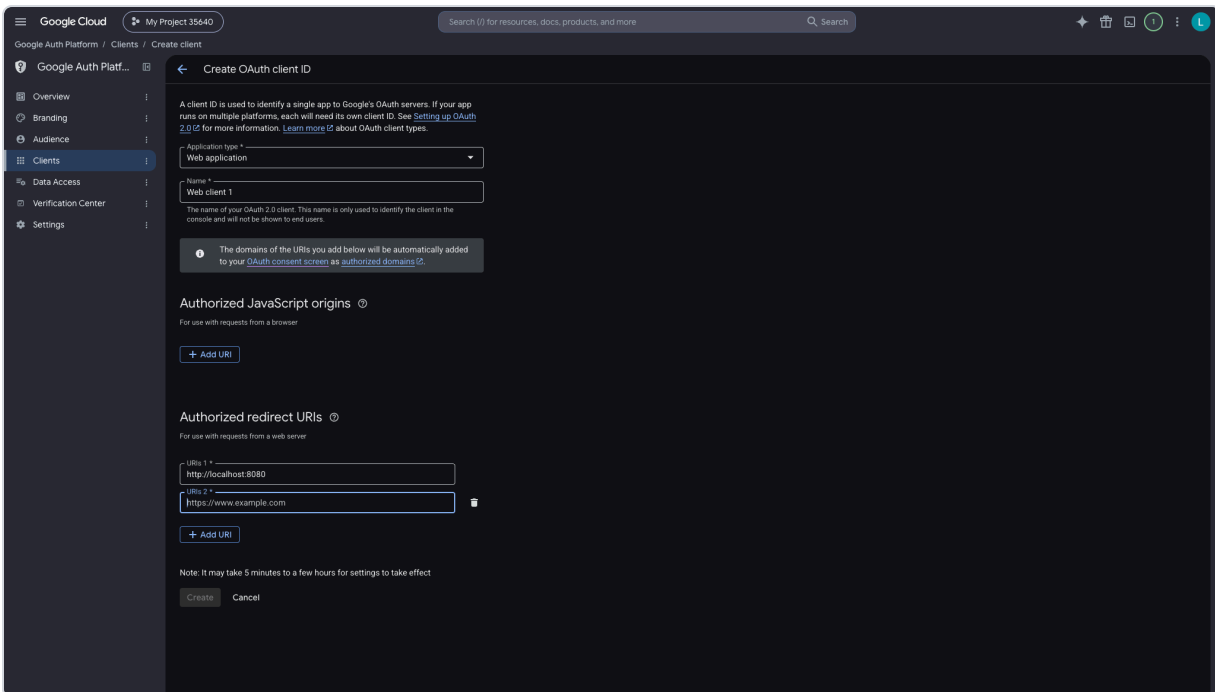
33 Type the first URI

Type `http://localhost:8080` exactly — no trailing slash, no path. Then click **+ Add URI** again.



34 Redirect URI 2

Click the new **URIs 2** box.



35 Type the second URI, then create

Type `http://localhost:3333/callback`. Then click the blue **Create** button.

- Both URIs matter: the sign-in can come back on either port, and a mismatch here is the #1 cause of a *redirect_uri_mismatch* error later.

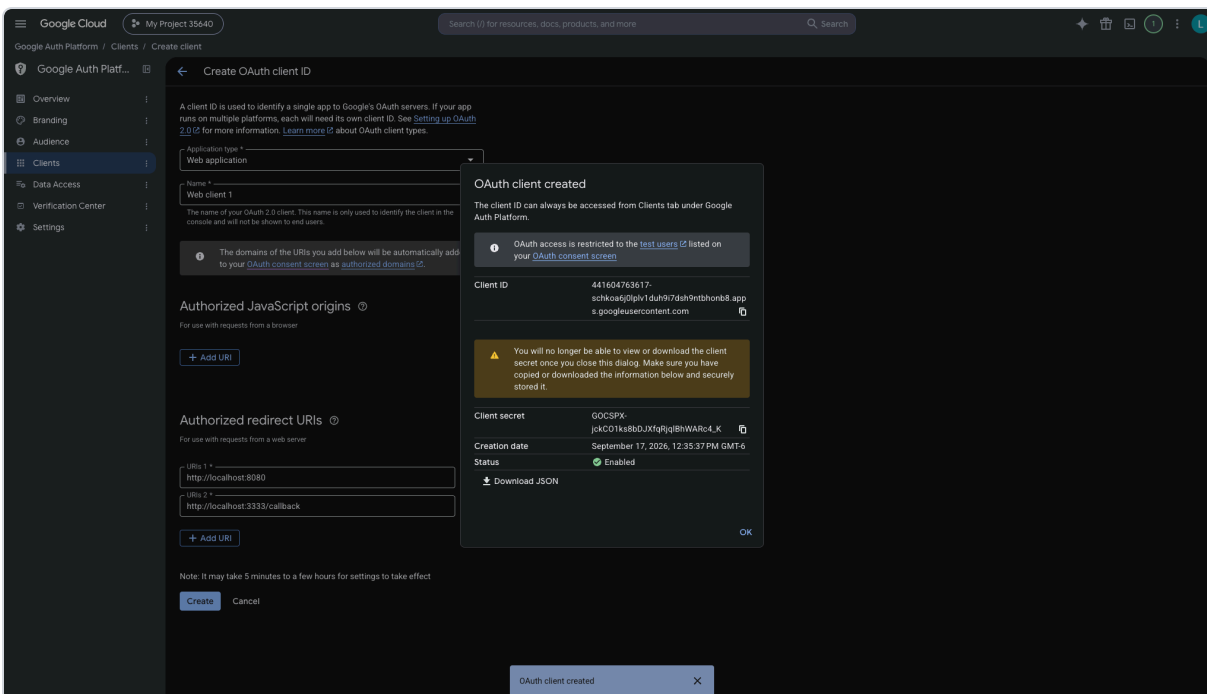
The screenshot shows the Google Cloud console interface for creating an OAuth client ID. The left sidebar contains navigation links for Overview, Branding, Audience, Clients, Data Access, Verification Center, and Settings. The main content area is titled 'Create OAuth client ID' and includes a brief explanation of client IDs. Below this, there are input fields for 'Application type' (set to 'Web application') and 'Name' (set to 'Web client 1'). A note states that the domains of the URIs added will be automatically added to the OAuth consent screen. There are two sections for adding URIs: 'Authorized JavaScript origins' and 'Authorized redirect URIs'. The 'Authorized redirect URIs' section contains two input fields: 'URIs 1' with the value 'http://localhost:8080' and 'URIs 2' with the value 'http://localhost:3333/callback'. At the bottom, there is a note about the time it may take for settings to take effect, and two buttons: 'Create' (highlighted in blue) and 'Cancel'.

Part 6 Finish the install

36 Download the client JSON

In the **OAuth client created** dialog, click **↓ Download JSON**. Then click **OK**.

- Don't close this dialog before downloading — the client secret can't be viewed again afterwards.
- The file lands in your **Downloads** folder, named something like `client_secret_44160...json`.



37 Return to Terminal and press Return

Switch back to the Terminal window from Step 1 (⌘-Tab). It's waiting on “Press Enter once you have downloaded the client JSON to pick it...” — press **Return**.

38 Pick the file you just downloaded

A macOS file picker opens. Go to **Downloads**, click the `client_secret_....json` file you just saved, and click **Open**.

39 Setup complete

Nothing to click — just confirm you see “**Setup complete.**” at the bottom. The installer wrote `client_secret_...json`, created the `rmx` command, and added it to your PATH.

- The yellow *warn:* lines are expected — they explain the Web-application client choice from Step 30.

```
-core#2.0.*,#2.1.*,#2.2.*,#2.3.0,<3.0.0,>1.31.5→google-api-python-client→youtube-upload=0.8.0 (2026.7.22)
Requirement already satisfied: cffi>2.0.0 in ./youtube-upload/venv/lib/python3.14/site-packages (from cryptography>41.0.5→google-auth#2.24.0
#2.25.0,<3.0.0,>1.32.0→google-api-python-client→youtube-upload=0.8.0) (2.1.1)
Requirement already satisfied: pycparser in ./youtube-upload/venv/lib/python3.14/site-packages (from cffi>2.0.0→cryptography>41.0.5→google-a
uth#2.24.0,#2.25.0,<3.0.0,>1.32.0→google-api-python-client→youtube-upload=0.8.0) (3.0)
Requirement already satisfied: pyasn1<0.7.0,>0.6.1 in ./youtube-upload/venv/lib/python3.14/site-packages (from pyasn1-modules>0.2.1→google-au
th#2.24.0,#2.25.0,<3.0.0,>1.32.0→google-api-python-client→youtube-upload=0.8.0) (0.6.4)
Requirement already satisfied: rsa>3.1.4 in ./youtube-upload/venv/lib/python3.14/site-packages (from oauth2client→youtube-upload=0.8.0) (4.9.
1)
Requirement already satisfied: six>1.6.1 in ./youtube-upload/venv/lib/python3.14/site-packages (from oauth2client→youtube-upload=0.8.0) (1.17
.0)
Requirement already satisfied: python-utils>3.8.1 in ./youtube-upload/venv/lib/python3.14/site-packages (from progressbar2→youtube-upload=0.8
.0) (4.0.1)
Building wheels for collected packages: youtube-upload
  Building wheel for youtube-upload (pyproject.toml) ... done
  Created wheel for youtube-upload: filename=youtube_upload-0.8.0-py3-none-any.whl size=13912 sha256=b8588deb491fed8196d5a7e12a5ce96003b15ab756f
f9825e1c245a30cfd7d3
  Stored in directory: /private/var/folders/wm/zmvbf12d5zd4brb97rjix_v4000gn/T/pip-ephem-wheel-cache-Sop7_y52/wheels/b2/7d/00/3752c09944d9f9202
b170fafb7a85f8017f93403dd1d0a8017
Successfully built youtube-upload
Installing collected packages: youtube-upload
Successfully installed youtube-upload-0.8.0
⇒ youtube-upload verified working.
⇒ Patching youtube-upload to use the loopback OAuth flow
⇒ OAuth flow patched.
⇒ Opened https://console.cloud.google.com/apis/dashboard in your browser.

Press Enter once you have downloaded the client JSON to pick it...
warn: That is a "Web application" OAuth client.
warn: Converting it works, but Google still only accepts redirect URIs you
warn: registered in the console, so sign-in succeeds only on a loopback
warn: port listed there (http://localhost:PORT, no path). A "Desktop app"
warn: client accepts any loopback port and needs no such bookkeeping.
⇒ Wrote /Users/Lawrence/youtube-upload/client_secrets.json
⇒ client id : 441604763617-schkoagj0lplv1duh9i7dsh9ntbthob8.apps.googleusercontent.com
⇒ type : web
⇒ Writing /Users/Lawrence/.local/bin/rmx
⇒ Put /Users/Lawrence/.local/bin on PATH in /Users/Lawrence/.zshrc
⇒ Setup complete.
Lawrence@Lawrences-MacBook-Air ~ %
```

40 Close Terminal, then reopen it

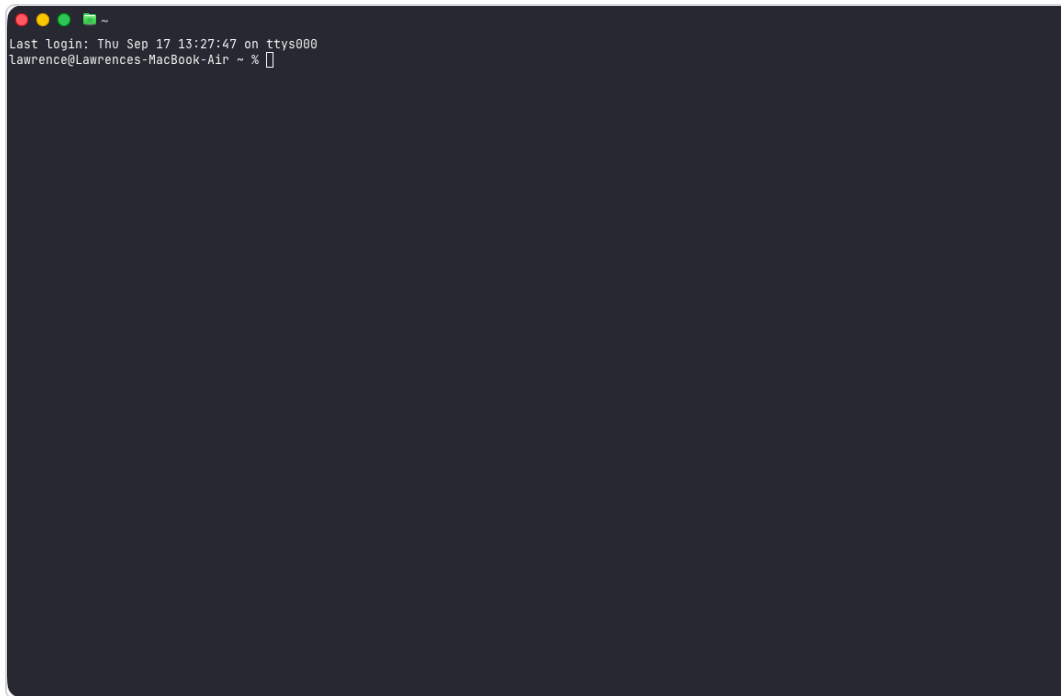
Quit Terminal fully (⌘-Q), then open it again. This reloads your `.zshrc` so the new `rmx` command is found.

- Without the restart you'll get *zsh: command not found: rmx.*

Part 7 How to use it

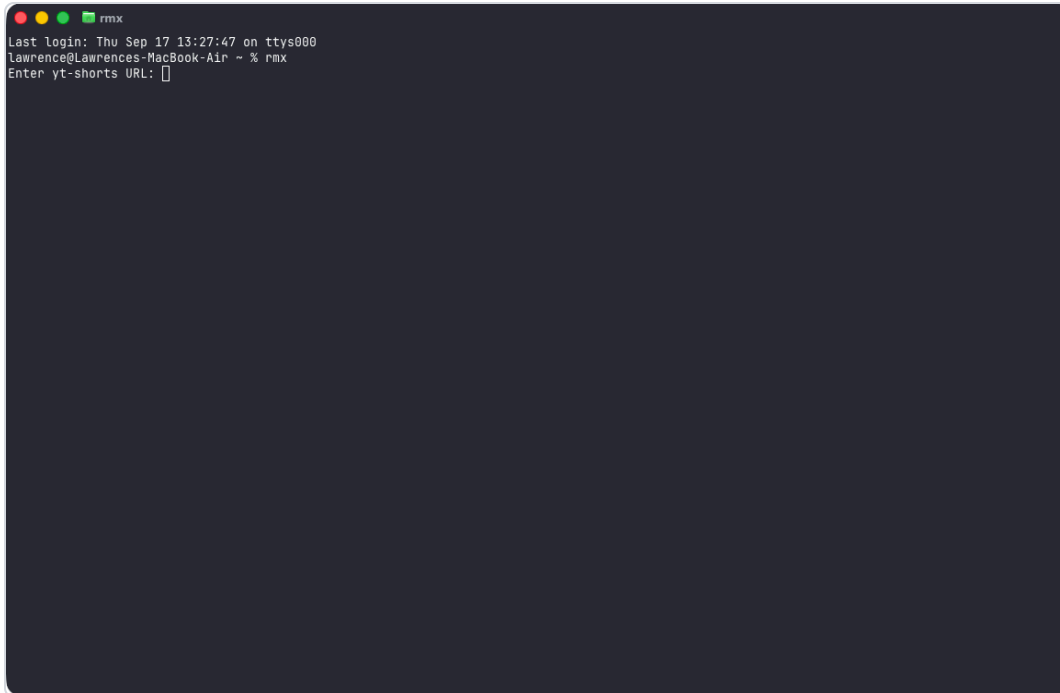
41 Fresh Terminal window

You're back at a clean prompt. Setup is done — everything below is normal day-to-day use.



42 Run rmx

Type `rmx` and press **Return**. It answers with **Enter yt-shorts URL:**

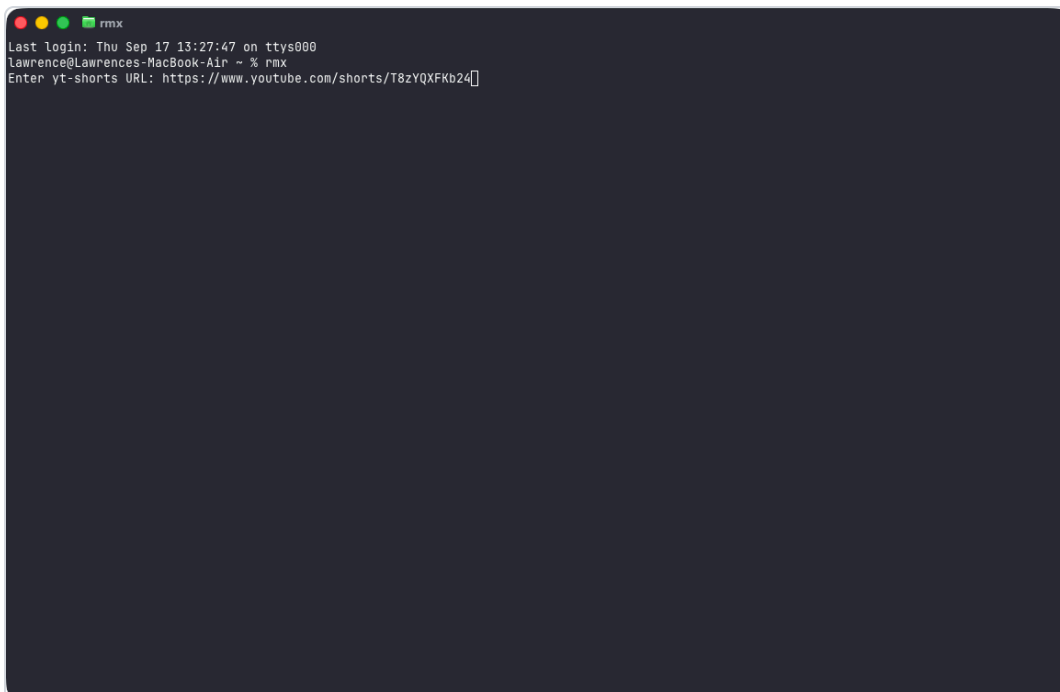


```
rmx
Last login: Thu Sep 17 13:27:47 on ttys000
lawrence@Lawrences-MacBook-Air ~ % rmx
Enter yt-shorts URL: 
```

43 Paste the video URL

Paste the YouTube URL you want to repost (⌘-V) and press **Return**.

- Example: `https://www.youtube.com/shorts/T8zYQXFKb24`



```
rmx
Last login: Thu Sep 17 13:27:47 on ttys000
lawrence@Lawrences-MacBook-Air ~ % rmx
Enter yt-shorts URL: https://www.youtube.com/shorts/T8zYQXFKb24
```

44 Approve in the browser

A Google OAuth screen opens automatically. Click your Google account, then on the “*Google hasn’t verified this app*” screen click **Advanced** → **Go to youtube-uploader (unsafe)**, then **Continue / Allow**.

- That warning is expected — it's your own unpublished app, in testing mode.
- You only authorize once; after that the token is cached.

45 Select your channel

If the account has more than one channel, click the channel you want the repost to land on.

46 Done

Return to Terminal and watch it download, remix, and upload. When it finishes, the video is on your channel.

47 Every time after this

Just open Terminal, type `rmx`, press **Return**, and paste the next URL. No setup needed again.

One last thing — be respectful of copyright.

Only repost material you own or are licensed to use. Reuploading someone else’s video without permission can get strikes on your channel, or get it terminated.